.UBAfiuba
FACULTAD DE INGENIERÍA

Teoría de Algoritmos
(75.29) Curso Buchwald - Genender

Trabajo Práctico 2
Programación Dinámica For The Win

10 de octubre de 2026

Juan David Chaparro
111481

1. Introducción

Es un juego de dos personas donde hay una fila de n monedas. En cada turno, el jugador solo puede sacar una de las dos monedas de los extremos: la primera o la ultima. Los turnos se alternan hasta que no quedan monedas, y gana el que junta mas valor, no el que tiene mas monedas.

A diferencia del TP1, ahora Mateo juega solo y usa un algoritmo greedy: en cada turno saca la moneda mas grande entre las dos puntas.

Sophia empieza siempre y quiere juntar el maximo valor posible. Para eso usa programacion dinamica. Aun asi, no siempre puede ganar. Por ejemplo, con $[1, 10, 5]$, haga lo que haga, Mateo se lleva el 10 y gana.

2. Analisis del problema

Mateo siempre juega igual, ya sabemos que moneda va a sacar en cada turno. Entonces la unica que decide es Sophia, y en cada turno tiene solo dos opciones: sacar la de la izquierda o la de la derecha.

Despues de que juegan Sophia y Mateo, lo que queda siempre son monedas seguidas, desde una posicion i hasta una posicion j. Es el mismo problema, pero con menos monedas. Ademas, a la misma fila se puede llegar de distintas formas. Por ejemplo, sacar primero la izquierda y despues la derecha deja lo mismo que hacerlo al reves. Por eso conviene resolver cada fila una sola vez y guardar el resultado en una tabla. eso es PD.

2.1. Por que no sirve greedy para Sophia

Si Sophia tambien sacara siempre la moneda mas grande, le podria dejar a Mateo una moneda buena. Por ejemplo, con $[3, 9, 1, 2]$:

Sophia saca	Despues	Sophia	Mateo
El 3 (la mas grande)	Mateo saca el 9, Sophia el 2, Mateo el 1	5	10
El 2	Mateo saca el 3, Sophia el 9, Mateo el 1	11	4

Si saca la mas grande, pierde. Si saca la mas chica, Mateo tiene que sacar el 3 y el 9 queda para Sophia. Para darse cuenta de esto hay que mirar las jugadas que vienen despues.

3. Ecuacion de recurrencia

Llamamos $m_0, m_1, \ldots, m_{n-1}$ a las monedas y definimos:

$OPT(i, j) =$ lo maximo que junta Sophia con las monedas de i a j, cuando le toca a ella.

Si Sophia saca la de la izquierda, quedan las monedas de $i+1$ a j y juega Mateo. Si saca la de la derecha, quedan las de i a $j-1$ y juega Mateo.

Para saber que le queda a Sophia despues de que Mateo juega con las monedas de a a b, usamos:

$$RESTO(a, b) = \begin{cases} 0 & \text{si } a > b \text{ (no quedan monedas)} \\ OPT(a+1, b) & \text{si } m_a \geq m_b \text{ (Mateo saca la izquierda)} \\ OPT(a, b-1) & \text{si } m_a < m_b \text{ (Mateo saca la derecha)} \end{cases}$$

Y la ecuacion queda:

$$OPT(i,j) = \begin{cases} 0 & \text{si } i > j \\ \max\left(m_i + RESTO(i+1,j),\ m_j + RESTO(i,j-1)\right) & \text{si } i \leq j \end{cases}$$

El resultado que buscamos es $OPT(0, n-1)$, que es la fila completa.

Si las dos puntas valen lo mismo, tomamos que Mateo saca la de la izquierda. Se lleva lo mismo, pero cambia lo que queda en la fila.

3.1. Ejemplo

Con $[3, 9, 1, 2]$, la tabla queda asi. La fila es i (donde empieza) y la columna es j (donde termina):

	$j=0$	$j=1$	$j=2$	$j=3$
$i=0$	3	9	4	**11**
$i=1$	-	9	9	10
$i=2$	-	-	1	2
$i=3$	-	-	-	2

La tabla se llena empezando por las filas de una moneda, despues las de dos, y asi. Para la casilla final:

- Si Sophia saca el 3, Mateo saca el 9 y quedan $[1, 2]$, que vale $OPT(2,3) = 2$. Total: $3+2 = 5$.
- Si Sophia saca el 2, Mateo saca el 3 y quedan $[9, 1]$, que vale $OPT(1,2) = 9$. Total: $2+9 = 11$.

El maximo es 11.

4. Demostracion

Queremos ver que $OPT(i,j)$ es de verdad lo maximo que puede juntar Sophia con las monedas de i a j, si Mateo juega greedy. Lo hacemos por induccion en la cantidad de monedas $k = j - i + 1$.

4.1. Casos base

- Si $k = 0$ no hay monedas, asi que Sophia no junta nada. La ecuacion da $OPT(i,j) = 0$, que es correcto.
- Si $k = 1$ hay una sola moneda y Sophia se la lleva. La ecuacion da $\max(m_i + 0,\ m_i + 0) = m_i$, que es correcto.

4.2. Paso inductivo

Suponemos que la ecuacion da el maximo para cualquier fila con menos de k monedas, y lo vemos para una fila de k monedas.

Cualquier forma de jugar de Sophia empieza sacando la moneda de la izquierda o la de la derecha. Miramos cada caso.

Si saca la izquierda: se lleva m_i y quedan las monedas de $i+1$ a j. Ahora juega Mateo, y como siempre saca la mas grande, ya sabemos que moneda saca y que fila queda despues. Esa fila tiene menos de k monedas. Haga lo que haga Sophia en esa fila, no puede juntar mas que su OPT, porque por hipotesis inductiva ese es el maximo. Y ademas ese valor se puede alcanzar. Entonces lo mejor que puede hacer Sophia empezando por la izquierda es exactamente $m_i + RESTO(i+1, j)$.

Si saca la derecha: pasa lo mismo. Lo mejor que puede hacer empezando por la derecha es $m_j + RESTO(i, j-1)$.

Como cualquier forma de jugar empieza por uno de esos dos lados, ninguna puede juntar mas que el mayor de los dos valores. Y ese mayor se puede alcanzar eligiendo el lado que lo da. Entonces:

$$OPT(i,j) = \text{máx}\left(m_i + RESTO(i+1,j),\ m_j + RESTO(i,j-1)\right)$$

es el maximo que puede juntar Sophia. Por induccion, vale para cualquier cantidad de monedas,

5. Algoritmo

El algoritmo esta en el archivo `dinamica.py` y tiene tres funciones:

- `resto`: hace la jugada de Mateo. Compara las dos puntas, saca la mas grande (si son iguales, la de la izquierda) y devuelve lo que la tabla dice que puede juntar Sophia con lo que queda.
- `calcular`: arma dos tablas. En `tabla[inicio][fin]` guarda lo maximo que junta Sophia con las monedas de `inicio` a `fin`, y en `eleccion[inicio][fin]` guarda si le conviene sacar la izquierda o la derecha. Las llena empezando por las filas de una moneda, despues las de dos, y asi, porque cada casilla usa filas mas cortas que ya tienen que estar calculadas.
- `jugar`: recorre la partida desde la fila completa. En el turno de Sophia mira la tabla `eleccion`, y en el de Mateo compara las puntas. Va guardando cada jugada.

O sea, `calcular` dice cuanto puede juntar Sophia, y `jugar` dice que monedas tiene que sacar.

```python
def resto(monedas, tabla, inicio, fin):
    if inicio > fin:
        return 0

    if monedas[inicio] >= monedas[fin]:
        inicio = inicio + 1
    else:
        fin = fin - 1

    if inicio > fin:
        return 0
    return tabla[inicio][fin]


def calcular(monedas):
    n = len(monedas)
    if n == 0:
        return [], []

    tabla = []
    eleccion = []
    for fila in range(n):
        tabla.append([0] * n)
        eleccion.append([""] * n)

    for largo in range(1, n + 1):
        for inicio in range(n - largo + 1):
            fin = inicio + largo - 1

            tomar_izq = monedas[inicio] + resto(monedas, tabla, inicio + 1, fin)
            tomar_der = monedas[fin] + resto(monedas, tabla, inicio, fin - 1)

            if tomar_izq >= tomar_der:
                tabla[inicio][fin] = tomar_izq
                eleccion[inicio][fin] = "izq"
            else:
                tabla[inicio][fin] = tomar_der
```

```
                eleccion[inicio][fin] = "der"

    return tabla, eleccion


def jugar(monedas, eleccion):
    inicio = 0
    fin = len(monedas) - 1
    turno_sophia = True
    jugadas = []

    while inicio <= fin:
        if turno_sophia:
            jugador = "Sophia"
            lado = eleccion[inicio][fin]
        else:
            jugador = "Mateo"
            if monedas[inicio] >= monedas[fin]:
                lado = "izq"
            else:
                lado = "der"

        if lado == "izq":
            jugadas.append((jugador, "primera", monedas[inicio]))
            inicio = inicio + 1
        else:
            jugadas.append((jugador, "ultima", monedas[fin]))
            fin = fin - 1

        turno_sophia = not turno_sophia

    return jugadas
```

6. Complejidad

6.1. Tiempo

- **Llenar la tabla:** hay una casilla por cada par (i, j) con $i \leq j$, o sea $\frac{n(n+1)}{2}$ casillas. Cada una se calcula con unas pocas sumas y comparaciones, y buscando valores que ya estan en la tabla, asi que cada casilla cuesta $O(1)$. En total es $O(n^2)$.
- **Reconstruir las jugadas:** la funcion `jugar` da una vuelta por cada moneda, asi que es $O(n)$.

El total es $O(n^2) + O(n) = O(n^2)$.

6.2. Memoria

Usamos dos tablas de $n \times n$: `tabla` y `eleccion`. Eso es $O(n^2)$ de memoria.

6.3. Comparacion con fuerza bruta

Si probaramos todas las formas de jugar de Sophia, como en cada turno tiene dos opciones y juega mas o menos $n/2$ veces, habria que probar unas $2^{n/2}$ partidas. Con 40 monedas ya son mas de un millon. La programacion dinamica evita esto porque cada fila se calcula una sola vez.

7. Pruebas

Para comprobar que el algoritmo da bien hicimos el archivo `pruebas.py`, que prueba lo siguiente:

- **Comparacion con fuerza bruta:** hicimos una funcion recursiva que prueba todas las jugadas posibles de Sophia, con Mateo jugando greedy. La comparamos con la programacion dinamica en 500 filas al azar de 1 a 12 monedas, con valores entre 1 y 20. Usamos valores chicos para que haya muchos empates.
- **Ejemplos conocidos:** con $[3, 9, 1, 2]$ da 11 y con $[1, 10, 5]$ da 6.
- **Casos borde:** una sola moneda (Sophia se la lleva), dos monedas (Sophia saca la mas grande), todas las monedas iguales y la fila vacia.
- **Las jugadas cuadran:** lo que suma Sophia en la lista de jugadas da lo mismo que la tabla, y hay tantas jugadas como monedas.
- **Desempate de Mateo:** con $[100, 5, 9, 1, 5]$, Sophia saca el 100 y Mateo queda con dos 5 en las puntas. Como saca el de la izquierda, Sophia termina con 110. Si sacara el de la derecha, terminaria con 106.

Todas las pruebas pasaron. Esto no reemplaza la demostracion, pero la respalda.

8. Mediciones de tiempos

Para ver si el algoritmo de verdad tarda $O(n^2)$, medimos cuanto tarda con filas de distintos tamanos. Las monedas se generaron al azar con el modulo `random` de Python.

9. Conclusiones

Como Mateo siempre juega igual, la unica que decide es Sophia. Eso permite partir el problema en filas mas chicas y resolverlo con programacion dinamica.

Vimos que greedy no sirve para Sophia: sacar siempre la moneda mas grande le puede dejar a Mateo una moneda mejor. La programacion dinamica, en cambio, mira las jugadas que vienen despues y le da el maximo posible.

Aun asi, Sophia no siempre puede ganar. Con filas como $[1, 10, 5]$, Mateo gana haga lo que haga ella.

El algoritmo tarda $O(n^2)$ y usa $O(n^2)$ de memoria, mucho mejor que probar todas las jugadas, que crece de forma exponencial.